ISEN
ALL IS DIGITAL!
OUEST

yncréa

Electronic Practical Work

Microcontrollers Security

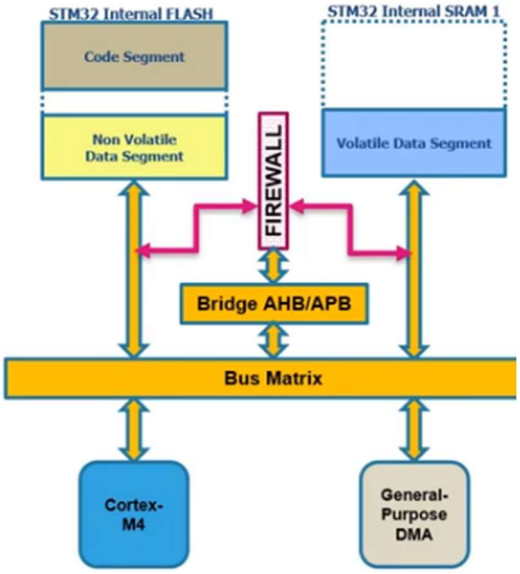
Firewall

Jean-Jacques Meneu

Firewall allows you to create a security enclave to securely execute a program. A typical program uses 3 memory areas:

- The code section in the Flash
- The initialized variables section in the Flash
- The variables section in the SRAM

The Firewall is a device that controls communications on the AHB bus. Therefore, it must be configured like any device when initializing the program.



In order to demonstrate how the firewall works, we call a function behind the firewall that changes the state of the LED every 500ms. This function cannot be called directly (otherwise a reset is generated) but via a function that controls access to the program behind the firewall.

Work to be done

- ✓ Read the Firewall section in the Reference Manual and understand the seven registers to configure
- ✓ The Firewall cannot be set up in the Graphical Interface called CubeMX. The installation of the drivers must be done manually. In the repository, copy/paste the Firewall drivers
- ✓ Enable the Firewall in the stm32L4xx_hal_conf.h file
- ✓ The firewall drivers are not mandatory and the registers can be configured directly by pointers as a 2nd option.
- ✓ Our program does not need a secure data section. In the linker script, make a partition of the following memory:
  - A memory area called SecureFlash in read and execute, which starts at address 0x080FC004 and is 16K in size. Reduce the size of the unsecured Flash to 1008K
  - Create a SecureSRAM area at address 0x2000FE00 of size 512 bytes
  - Note: these addresses are chosen arbitrarily. Different addresses can be chosen
- ✓ In the SECTIONS section, the linker script describes how the different areas should be filled. In order to put the desired functions in the SecureFlash section, we call a section

mysection (this name can be changed. The chosen name will be reused in the C code as an attribute (see below)). Copy the code below into the SECTIONS section

```
.mysection :
{
 . = ALIGN(4);
 *(.mysection*)
 . = ALIGN(4);
} > SecureFlash
```

Do the same with SRAM

```
.securedata :
{
 *(.securedata)
 *(.securedata*)
} > SecureSRAM
```

- ✓ The initialization and launch procedure are set up with the following steps

Below is the initialization procedure to follow:

1. Configure the RCC to enable the clock to the Firewall module
2. Configure the RCC to enable the clock of the system configuration registers
3. Set the base address and length of each segment (CSSA, CSL, NVDSSA, NVDSL, VDSSA, VDSL registers)
4. Set the configuration register of the Firewall (FW_CR register)
5. Enable the Firewall clearing the FWDIS bit in the system configuration register.

- ✓ Program the initialization procedure. If the firewall library is installed, the following code can be used

```
FIREWALL_InitTypeDef fw_init;

fw_init.CodeSegmentStartAddress = 0x080FC000;
fw_init.CodeSegmentLength = 0x2000;

fw_init.NonVDataSegmentStartAddress = 0X080FE000;
fw_init.NonVDataSegmentLength = 0x200;

fw_init.VDataSegmentStartAddress = 0x2000FE00;
fw_init.VDataSegmentLength = 0x200;

HAL_FIREWALL_Config(&fw_init);

HAL_FIREWALL_EnableFirewall();
```

- ✓ Write a Protected_function function that takes no arguments. This function must be in the mysection section that we defined in the linker script. In C, this is how the function must be declared.
- ✓ void __attribute__((__section__(".mysection"))) Protected_function(void)
- ✓ Read the PreARM functionality
- ✓ In the Protected_function function:
  - Disable the PreARM thanks to the macro `__HAL_FIREWALL_PREARM_DISABLE();`
  - Call the toggleLED function (defined below)
  - Enable the PreARM thanks to the macro `__HAL_FIREWALL_PREARM_ENABLE();`
- ✓ Write a toggleLED function that changes the state of the diode on port A pin 5
- ✓ In order to avoid program execution outside the enclosed area, interrupts must be disabled before calling the function and re-enabled when the function returns. The code for the infinite loop is therefore as follows:

```
while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
    __disable_irq();
    Protected_function();
    //Toggle_led();
    __enable_irq();
    HAL_Delay(500);

}
```

- ✓ Test the program. Try to launch the Toggle_led function directly. What is the consequence?
- ✓ Declare an uninitialized variable (named number_calls) in the protected area of SRAM and initiliaze it to zero before enabling the firewall.
- ✓ Increment this variable each time a call to the protected_function is performed
- ✓ Confirm the value of the variables cannot be accessed when the firewall is closed

The firewall protects the memory but has no effect about the internal registers. Let's imagine that the counter incrementation was written in assembly language with the following code:

```
__asm__ (
        "ldr r4, =number_calls\n"
        "ldr r5, [r4]\n"
        "add r5, #1\n"
        "str r5, [r4]\n"
        "ldr r6, =0xDEADBEEF");
```

- ✓ Replace the C code where the variable number_calls was incremented by the assembly code given above
- ✓ Check values of internal registers before and after the exit
- ✓ Where the information about the number of calls is available?
- ✓ What do you recommend to prevent this leak?